Algoritmos de optimización - Seminario

Nombre y Apellidos:

- Alberto Paparelli
- Raúl Sánchez Mondéjar

Url: https://colab.research.google.com/drive/1HbELuadO33_BsiZnb88AF35mMdJzIKo6

Problema:

> 2. Organizar los horarios de partidos de La Liga

Descripción del problema:

- Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.
- Los horarios disponibles se conocen a priori y son los siguientes:

Viernes	20
Sábado	12, 16, 18, 20
Domingo	12, 16, 18, 20
Lunes	20

- En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores (que tiene relación directa con la audiencia). Hay 3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C.
- Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos).

	Categoría A	Categoría B	Categoría C
Categoría A	2 Millones	1.3 Millones	1 Millon
Categoría B		0.9 Millones	0.75 Millones

- Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla.
- Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes.

	Viernes	Sábado	Domingo	Lunes
12h	-	0.55	0.45	-
16h	-	0.7	0.75	-
18h	-	0.8	0.85	-
20h	0.4	1	1	0.4

- Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Coincidencias	-%
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

- Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

Partido	Categorías	Horario	Base (Millones)	Ponderación	Base * Ponderación	Corrección Coincidencia
Celta - Real Madrid	B-A	V20	1.3	0.4	0.52	0.52
Valencia - R. Sociedad	B-A	S12	1.3	0.55	0.72	0.72
Mallorca - Eibar	C-C	S16	0.47	0.7	0.33	0.33
Athletic - Barcelona	B-A	S18	1.3	0.8	1.04	1.04
Leganés - Osasuna	C-C	S20	0.47	1	0.47	0.47
Villarreal - Granada	**B-C**	**D16**	**0.75**	**0.75**	**0.56**	**0.42** *(1)
Alavés - Levante	**B-B**	**D16**	**0.9**	**0.75**	**0.68**	**0.51** *(2)
Espanyol - Sevilla	B-B	D18	0.9	0.85	0.77	0.77
Betis - Valladolid	B-C	D20	0.75	1	0.75	0.75
Atlético - Getafe	B-B	L20	0.9	0.4	0.36	0.36
						TOTAL: 5.88

*(1) 0.42 = 0.56 x 0.75

*(2) 0.51 = 0.68 x 0.75

(*) La respuesta es obligatoria

* ¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

Datos:

- 10 partidos por jornada (20 equipos / 2)
- 10 franjas horarias disponibles: V20, S12, S16, S18, S20, D12, D16, D18, D20, L20

Sin restricciones, cada partido puede asignarse a cualquiera de las 10 franjas horarias, y se permite que varios partidos coincidan en la misma franja.

Esto es un problema de ASIGNACIÓN CON REPETICIÓN: a cada uno de los n partidos le asignamos una de las k franjas horarias posibles.

Por el principio multiplicativo:

- Partido 1 -> 10 opciones
- Partido 2 -> 10 opciones
- ...
- Partido 10 -> 10 opciones

Total = k^n = 10^10

In [82]:
```
N_PARTIDOS = 10   # partidos por jornada
K_FRANJAS  = 10   # franjas horarias disponibles (índices 0-9)

# Total de asignaciones posibles (asignación con repetición)
total_posibilidades = K_FRANJAS ** N_PARTIDOS

print(f"Partidos por jornada: {N_PARTIDOS}")
print(f"Franjas horarias disponibles: {K_FRANJAS}")
print(f"Total de posibilidades (sin restricciones): {K_FRANJAS}^{N_PARTIDOS} = {total_posibilidades:,}")
```

```
Partidos por jornada: 10
Franjas horarias disponibles: 10
Total de posibilidades (sin restricciones): 10^10 = 10,000,000,000
```

Respuesta:

Total de posibilidades (sin restricciones):

$$10^{10} = 10,000,000,000$$

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Restricciones: * Obligatoriamente debe haber al menos 1 partido el viernes. * Obligatoriamente debe haber al menos 1 partido el lunes.

Para contar las asignaciones válidas usamos el principio de inclusión-exclusión:

Sea:

- T = total sin restricciones = 10^{10}
- A = asignaciones donde **ningún** partido se juega el viernes → cada partido elige entre 9 slots = 9^{10}
- B = asignaciones donde **ningún** partido se juega el lunes → 9^{10}
- $A \cap B$ = asignaciones sin viernes **ni** lunes → cada partido elige entre 8 slots = 8^{10}

Aplicando inclusión-exclusión:

$$\text{Válidas} = T - |A| - |B| + |A \cap B| = 10^{10} - 2 \times 9^{10} + 8^{10}$$

In [83]:
```
total = 10**10           # sin restricciones
sin_viernes = 9**10      # ningún partido el viernes (9 slots disponibles)
sin_lunes = 9**10        # ningún partido el lunes (9 slots disponibles)
sin_ambos = 8**10        # ningún partido el viernes ni el lunes (8 slots)

# Inclusión-exclusión: al menos 1 viernes Y al menos 1 lunes
validas = total - sin_viernes - sin_lunes + sin_ambos

print(f"Total sin restricciones:          {total:>15,}")
print(f"Sin viernes (9^10):               {sin_viernes:>15,}")
print(f"Sin lunes (9^10):                 {sin_lunes:>15,}")
print(f"Sin viernes ni lunes (8^10):      {sin_ambos:>15,}")
print(f"Total con restricciones:          {validas:>15,}")
```

```
Total sin restricciones:          10,000,000,000
Sin viernes (9^10):                3,486,784,401
Sin lunes (9^10):                  3,486,784,401
Sin viernes ni lunes (8^10):       1,073,741,824
Total con restricciones:           4,100,173,022
```

Respuesta:

Teniendo en cuenta la restricción de al menos 1 partido el viernes y 1 el lunes, el número de asignaciones válidas es:

$$10^{10} - 2 \times 9^{10} + 8^{10} = 4,100,173,022$$

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.
(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, arguméntalo)

Respuesta: Vector (lista Python)

Representación:

Utilizamos una **lista de 10 enteros** donde la posición `i` representa el partido `i` y el valor almacenado indica el índice de la franja horaria asignada (0-9).

```
solucion = [3, 0, 5, 7, 4, 6, 6, 8, 9, 1]
              ^  ^  ^   ...
           P0->S18  P1->V20  P2->D12 ...
```

Mapeo de franjas horarias:

Índice	Franja
0	V20
1	S12
2	S16
3	S18
4	S20
5	D12
6	D16
7	D18
8	D20
9	L20

Argumentación:

1. **Consistencia con lo visto en clase:** Es la misma estructura utilizada en el problema de N-Reinas (AG1): un vector donde la posición representa la entidad a asignar y el valor representa la asignación. Además, el manual y las actividades guiadas usan vectores para problemas de asignación.
2. **Eficiencia computacional:** Acceso O(1) por índice, tamaño fijo conocido (n=10), fácil generación de vecinos (cambiar un valor en una posición), y compatible con fuerza bruta (recorrer las 10^{10} combinaciones mediante conteo en base 10).
3. **Verificación de restricciones simple:** Contar coincidencias por franja con un conteo manual mediante un vector auxiliar de tamaño k (O(n)), verificar la presencia de viernes (valor 0) o lunes (valor 9) con una simple comparación de valores en el vector.

Alternativa descartada, Diccionario `{franja: [partidos]}`:

La perspectiva inversa (agrupar partidos por franja) facilita calcular coincidencias, pero complica la generación de soluciones y la iteración del espacio de búsqueda. Esta vista se puede derivar fácilmente del vector cuando se necesite, por lo que no justifica usarla como estructura principal.

In [84]:

```python
# ----------------------------------------------------------------------------
# Definición de la representación
# ----------------------------------------------------------------------------

N_PARTIDOS = 10   # partidos por jornada
N_FRANJAS  = 10   # franjas horarias disponibles (índices 0-9)

# Mapeo índice → etiqueta legible de franja horaria
IDX_A_FRANJA = {
    0: "V20", 1: "S12", 2: "S16", 3: "S18", 4: "S20",
    5: "D12", 6: "D16", 7: "D18", 8: "D20", 9: "L20",
}

# Índices especiales para restricciones
IDX_VIERNES = 0   # V20
IDX_LUNES   = 9   # L20

# Ejemplo: jornada del enunciado
# P0=Celta-RMadrid, P1=Valencia-RSociedad, P2=Mallorca-Eibar,
# P3=Athletic-Barcelona, P4=Leganés-Osasuna, P5=Villarreal-Granada,
# P6=Alavés-Levante, P7=Espanyol-Sevilla, P8=Betis-Valladolid,
# P9=Atlético-Getafe
partidos = [
    "Celta-RMadrid", "Valencia-RSociedad", "Mallorca-Eibar",
    "Athletic-Barcelona", "Leganés-Osasuna", "Villarreal-Granada",
    "Alavés-Levante", "Espanyol-Sevilla", "Betis-Valladolid",
    "Atlético-Getafe",
]

# Solución de ejemplo (la del enunciado)
# V20, S12, S16, S18, S20, D16, D16, D18, D20, L20
solucion_ejemplo = [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]

# ----------------------------------------------------------------------------
# Mostrar la asignación
# ----------------------------------------------------------------------------
print("Solución ejemplo (vector):", solucion_ejemplo)
print()
for i, franja_idx in enumerate(solucion_ejemplo):
    print(f"  P{i} ({partidos[i]:>22s}) -> franja {franja_idx} ({IDX_A_FRANJA[franja_idx]})")

# ----------------------------------------------------------------------------
# Verificación de restricciones
# ----------------------------------------------------------------------------
tiene_viernes = IDX_VIERNES in solucion_ejemplo
tiene_lunes = IDX_LUNES   in solucion_ejemplo
es_valida = tiene_viernes and tiene_lunes

print(f"\n¿Al menos 1 partido en viernes? {tiene_viernes}")
print(f"¿Al menos 1 partido en lunes?   {tiene_lunes}")
print(f"¿Solución válida?               {es_valida}")

# ----------------------------------------------------------------------------
# Coincidencias por franja (útil para penalizaciones)
# Conteo manual con vector auxiliar de tamaño k, O(n)
# ----------------------------------------------------------------------------
conteo = [0] * N_FRANJAS
for franja_idx in solucion_ejemplo:
    conteo[franja_idx] += 1

print("\nCoincidencias por franja:")
for franja_idx, cantidad in enumerate(conteo):
    if cantidad > 0:
```

```
        extra = f"  <- {cantidad - 1} coincidencia(s)" if cantidad > 1 else ""
        print(f"  {IDX_A_FRANJA[franja_idx]}: {cantidad} partido(s){extra}")
```

```
Solución ejemplo (vector): [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]

  P0 (         Celta-RMadrid) -> franja 0 (V20)
  P1 (    Valencia-RSociedad) -> franja 1 (S12)
  P2 (        Mallorca-Eibar) -> franja 2 (S16)
  P3 (    Athletic-Barcelona) -> franja 3 (S18)
  P4 (       Leganés-Osasuna) -> franja 4 (S20)
  P5 (    Villarreal-Granada) -> franja 6 (D16)
  P6 (        Alavés-Levante) -> franja 6 (D16)
  P7 (      Espanyol-Sevilla) -> franja 7 (D18)
  P8 (      Betis-Valladolid) -> franja 8 (D20)
  P9 (       Atlético-Getafe) -> franja 9 (L20)

¿Al menos 1 partido en viernes? True
¿Al menos 1 partido en lunes?   True
¿Solución válida?               True

Coincidencias por franja:
  V20: 1 partido(s)
  S12: 1 partido(s)
  S16: 1 partido(s)
  S18: 1 partido(s)
  S20: 1 partido(s)
  D16: 2 partido(s)  <- 1 coincidencia(s)
  D18: 1 partido(s)
  D20: 1 partido(s)
  L20: 1 partido(s)
```

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

Función objetivo:

La función objetivo calcula la **audiencia total** (en millones de espectadores) de una jornada, sumando la contribución de cada partido. La contribución de cada partido depende de tres factores multiplicativos:

$$f(S) = \sum_{i=0}^{n-1} \text{base}_i \times \text{pond}(S_i) \times \text{penalización}(\text{coinc}_i)$$

donde, para cada partido i:

- base_i: audiencia base según las categorías de los dos equipos (en millones, referida al horario S20).
- $\text{pond}(S_i)$: coeficiente de la franja horaria asignada al partido i (entre 0.4 y 1.0).
- $\text{coinc}_i = (\text{partidos en franja } S_i) - 1$: número de coincidencias en esa franja.
- $\text{penalización}(c)$: factor de reducción según la tabla de coincidencias (0 coinc → 1.0, 1 coinc → 0.75, ...).

Es un problema de maximización. Queremos encontrar la asignación de partidos a franjas horarias que maximice la audiencia total de la jornada.

Complejidad de evaluación: O(n) para evaluar una solución (un recorrido para contar coincidencias por franja + un recorrido para sumar audiencias).

In [85]:
```
# --- Datos del problema ---
# Audiencia base (millones) según categorías, simétrica
BASE_AUDIENCIA = {
    ("A","A"): 2.0,  ("A","B"): 1.3,  ("A","C"): 1.0,
    ("B","A"): 1.3,  ("B","B"): 0.9,  ("B","C"): 0.75,
    ("C","A"): 1.0,  ("C","B"): 0.75, ("C","C"): 0.47,
}

# Ponderación por franja horaria (índice 0-9)
PONDERACION = [0.4, 0.55, 0.7, 0.8, 1.0, 0.45, 0.75, 0.85, 1.0, 0.4]

# Factor de penalización por número de coincidencias en la misma franja
PENALIZACION = [1.0, 0.75, 0.55, 0.40, 0.30, 0.25, 0.22, 0.20, 0.20]

# Categorías de los equipos del ejemplo
categorias_ejemplo = [
    ("B","A"), ("B","A"), ("C","C"), ("B","A"), ("C","C"),
    ("B","C"), ("B","B"), ("B","B"), ("B","C"), ("B","B"),
]

def funcion_objetivo(solucion, categorias):
    """Calcula la audiencia total (en millones) de una asignación."""
    n = len(solucion)

    # Paso 1: contar partidos por franja, O(n)
    conteo = [0] * N_FRANJAS
    for franja in solucion:
        conteo[franja] += 1

    # Paso 2: sumar audiencia ponderada y penalizada, O(n)
    audiencia_total = 0.0
    for i in range(n):
        base  = BASE_AUDIENCIA[categorias[i]]
        pond  = PONDERACION[solucion[i]]
        coinc = conteo[solucion[i]] - 1          # coincidencias = partidos_en_franja - 1
        penal = PENALIZACION[coinc]
        audiencia_total += base * pond * penal

    return audiencia_total

# --- Validación con el ejemplo del enunciado ---
total = funcion_objetivo(solucion_ejemplo, categorias_ejemplo)
print(f"Audiencia total (ejemplo): {total:.4f} millones")
print(f"Valor esperado (enunciado, redondeado): 5.88 millones")
```

```python
# Desglose por partido
print("\nDesglose:")
conteo = [0] * N_FRANJAS
for f in solucion_ejemplo:
    conteo[f] += 1

for i in range(N_PARTIDOS):
    base  = BASE_AUDIENCIA[categorias_ejemplo[i]]
    pond  = PONDERACION[solucion_ejemplo[i]]
    coinc = conteo[solucion_ejemplo[i]] - 1
    penal = PENALIZACION[coinc]
    valor = base * pond * penal
    print(f"  P{i} {partidos[i]:>22s}  base={base:.2f}  pond={pond:.2f}"
          f"  coinc={coinc}  penal={penal:.2f}  -> {valor:.4f}")
```

```
Audiencia total (ejemplo): 5.8771 millones
Valor esperado (enunciado, redondeado): 5.88 millones

Desglose:
  P0          Celta-RMadrid  base=1.30  pond=0.40  coinc=0  penal=1.00  -> 0.5200
  P1     Valencia-RSociedad  base=1.30  pond=0.55  coinc=0  penal=1.00  -> 0.7150
  P2         Mallorca-Eibar  base=0.47  pond=0.70  coinc=0  penal=1.00  -> 0.3290
  P3     Athletic-Barcelona  base=1.30  pond=0.80  coinc=0  penal=1.00  -> 1.0400
  P4        Leganés-Osasuna  base=0.47  pond=1.00  coinc=0  penal=1.00  -> 0.4700
  P5     Villarreal-Granada  base=0.75  pond=0.75  coinc=1  penal=0.75  -> 0.4219
  P6         Alavés-Levante  base=0.90  pond=0.75  coinc=1  penal=0.75  -> 0.5063
  P7       Espanyol-Sevilla  base=0.90  pond=0.85  coinc=0  penal=1.00  -> 0.7650
  P8       Betis-Valladolid  base=0.75  pond=1.00  coinc=0  penal=1.00  -> 0.7500
  P9        Atlético-Getafe  base=0.90  pond=0.40  coinc=0  penal=1.00  -> 0.3600
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

Enfoque: Enumerar **todas** las k^n asignaciones posibles (donde $k = 10$ franjas y $n = 10$ partidos), verificar las restricciones para cada una, evaluar la función objetivo y quedarse con la asignación que maximice la audiencia total.

Pseudocódigo:

```
función FUERZA_BRUTA(categorías, n_partidos, n_franjas):
    mejor_solución <- nula
    mejor_audiencia  <- -∞
    solución ← [0, 0, ..., 0]          // primera combinación (n_partidos ceros)

    REPETIR:
        SI es_válida(solución):         // ¿tiene al menos 1 viernes y 1 lunes?
            audiencia <-> función_objetivo(solución, categorías)
            SI audiencia > mejor_audiencia:
                mejor_audiencia <- audiencia
                mejor_solución <- copia(solución)

        SI NO siguiente(solución, n_franjas):   // incrementar "odómetro" en base k
            SALIR                               // se recorrieron todas las combinaciones

    DEVOLVER mejor_solución, mejor_audiencia
```

Generación de combinaciones - conteo en base k (odómetro):

El vector solución se trata como un número en base k. La función `siguiente()` incrementa la última posición y propaga el acarreo hacia la izquierda, como un odómetro:

```
[0,0,0] -> [0,0,1] -> ... → [0,0,9] -> [0,1,0] → ... → [9,9,9] -> desbordamiento (fin)
```

Esto genera las k^n combinaciones sin necesidad de `itertools` ni recursión.

Inviabilidad para $n = 10$:

$10^{10} = 10{,}000{,}000{,}000$ combinaciones. Incluso evaluando 10^6 por segundo, se necesitarían $\approx 10{,}000$ segundos (≈ 2.8 horas). Es computacionalmente inviable.

Demostración con ejemplo reducido:

Para verificar la corrección del algoritmo, lo ejecutamos con los primeros 6 partidos de la jornada ejemplo: $10^6 = 1{,}000{,}000$ combinaciones, factible en pocos segundos.

In [86]:
```python
import time

def es_valida(solucion):
    """Restricciones: al menos 1 viernes (idx 0) y 1 lunes (idx 9)."""
    return IDX_VIERNES in solucion and IDX_LUNES in solucion

def siguiente(solucion, base):
    """Incrementa el vector en base k (odómetro). Devuelve False si desborda."""
    i = len(solucion) - 1
    while i >= 0:
        solucion[i] += 1
        if solucion[i] < base:
            return True
        solucion[i] = 0
        i -= 1
    return False  # desbordamiento → se recorrieron todas

def fuerza_bruta(categorias, n_partidos, n_franjas):
    """Enumera las k^n asignaciones por conteo en base k."""
    mejor_solucion = None
    mejor_audiencia = -1
    evaluadas = 0

    solucion = [0] * n_partidos      # primera combinación: [0,0,...,0]
    while True:
        if es_valida(solucion):
            evaluadas += 1
            audiencia = funcion_objetivo(solucion, categorias)
            if audiencia > mejor_audiencia:
```

```
                mejor_audiencia = audiencia
                mejor_solucion = solucion[:]
        if not siguiente(solucion, n_franjas):
            break

    return mejor_solucion, mejor_audiencia, evaluadas

# --- Demo con ejemplo reducido (6 partidos) ---
n_reducido = 6
cats_reducido = categorias_ejemplo[:n_reducido]

print(f"Fuerza bruta con {n_reducido} partidos "
      f"({N_FRANJAS}^{n_reducido} = {N_FRANJAS**n_reducido:,} combinaciones)")

t0 = time.time()
sol, aud, ev = fuerza_bruta(cats_reducido, n_reducido, N_FRANJAS)
t1 = time.time()

print(f"Soluciones válidas evaluadas: {ev:,}")
print(f"Tiempo: {t1-t0:.2f} s")
print(f"\nMejor audiencia: {aud:.4f} millones")
print(f"Mejor solución:  {sol}")
for i in range(n_reducido):
    print(f"  P{i} ({partidos[i]:>22s}) -> {IDX_A_FRANJA[sol[i]]}")

# --- Estimación para n=10 ---
factor = N_FRANJAS ** (N_PARTIDOS - n_reducido)
t_estimado = (t1 - t0) * factor
print(f"\nPara n={N_PARTIDOS}: {N_FRANJAS}^{N_PARTIDOS} = "
      f"{N_FRANJAS**N_PARTIDOS:,} combinaciones")
print(f"Tiempo estimado: {t_estimado:.0f} s "
      f"({t_estimado/3600:.1f} h) -> INVIABLE")
```

```
Fuerza bruta con 6 partidos (10^6 = 1,000,000 combinaciones)
Soluciones válidas evaluadas: 199,262
Tiempo: 0.36 s

Mejor audiencia: 4.6810 millones
Mejor solución:  [4, 7, 0, 8, 9, 3]
  P0 (         Celta-RMadrid) -> S20
  P1 (    Valencia-RSociedad) -> D18
  P2 (        Mallorca-Eibar) -> V20
  P3 (    Athletic-Barcelona) -> D20
  P4 (       Leganés-Osasuna) -> L20
  P5 (    Villarreal-Granada) -> S18

Para n=10: 10^10 = 10,000,000,000 combinaciones
Tiempo estimado: 3562 s (1.0 h) -> INVIABLE
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

El algoritmo de fuerza bruta tiene tres operaciones principales que se ejecutan para cada una de las k^n combinaciones:

Operación	Coste por llamada	Llamadas
`siguiente()` : incrementar odómetro	$O(n)$ peor caso, $O(1)$ amortizado	k^n
`es_valida()` : buscar viernes y lunes en el vector	$O(n)$	k^n
`funcion_objetivo()` : contar coincidencias + sumar audiencias	$O(n)$	$\leq k^n$ (solo válidas)

Complejidad temporal:

Cada iteración del bucle realiza trabajo $O(n)$ (validación + evaluación + avance del odómetro), y hay k^n iteraciones:

$$T(n,k) = O(n \cdot k^n)$$

Con los valores del problema ($k = n = 10$):

$$T(10,10) = O(10 \cdot 10^{10}) = O(10^{11})$$

La complejidad es **exponencial** en n: El factor dominante es k^n, que crece mucho más rápido que el factor lineal n.

Complejidad espacial:

$$S(n,k) = O(n+k)$$

- $O(n)$: el vector solución actual + la mejor solución guardada.
- $O(k)$: el vector auxiliar de conteo por franja dentro de `funcion_objetivo`.
- No se almacenan todas las combinaciones en memoria; se generan una a una (generación implícita).

Clase de complejidad:

El algoritmo pertenece a la clase $O(k^n)$ tiempo exponencial. Es intratable para valores moderados de n y k, lo que motiva la búsqueda de algoritmos más eficientes (backtracking, metaheurísticas como el Recocido Simulado).

In [87]:
```
# --- Complejidad del algoritmo de fuerza bruta ---
# T(n,k) = O(n · k^n)   |   S(n,k) = O(n + k)

print("Crecimiento de la complejidad temporal T(n,k) = n · k^n")
print(f"{'n':>3s} {'k':>3s} {'k^n':>18s} {'n·k^n':>20s}")
print("-" * 48)

k = N_FRANJAS
for n in range(1, N_PARTIDOS + 1):
    comb = k ** n
    ops  = n * comb
    print(f"{n:>3d} {k:>3d} {comb:>18,} {ops:>20,}")

# Verificación empírica con los tiempos medidos en cell-23
print(f"\n--- Verificación empírica (cell-23) ---")
t_medido = t1 - t0  # tiempo medido para n=6
```

```
ops_n6 = n_reducido * (N_FRANJAS ** n_reducido)
ops_n10 = N_PARTIDOS * (N_FRANJAS ** N_PARTIDOS)
print(f"n=6:  {ops_n6:,} operaciones → {t_medido:.2f} s medidos")
print(f"n=10: {ops_n10:,} operaciones → factor ×{ops_n10/ops_n6:,.0f} respecto a n=6")
print(f"      Tiempo estimado: {t_medido * (ops_n10/ops_n6):.0f} s "
      f"({t_medido * (ops_n10/ops_n6) / 3600:.1f} h)")
```

```
Crecimiento de la complejidad temporal T(n,k) = n · k^n
  n   k                k^n                n·k^n
--------------------------------------------------
  1  10                 10                   10
  2  10                100                  200
  3  10              1,000                3,000
  4  10             10,000               40,000
  5  10            100,000              500,000
  6  10          1,000,000            6,000,000
  7  10         10,000,000           70,000,000
  8  10        100,000,000          800,000,000
  9  10      1,000,000,000        9,000,000,000
 10  10     10,000,000,000      100,000,000,000

--- Verificación empírica (cell-23) ---
n=6:  6,000,000 operaciones → 0.36 s medidos
n=10: 100,000,000,000 operaciones → factor ×16,667 respecto a n=6
      Tiempo estimado: 5936 s (1.6 h)
```

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta: Recocido Simulado (Simulated Annealing)

Idea clave: Partir de una solución inicial aleatoria y mejorarla iterativamente explorando el espacio de búsqueda de forma *global*. A diferencia del algoritmo voraz, el recocido simulado **acepta probabilísticamente soluciones peores** para escapar de óptimos locales y explorar otras regiones del espacio.

Analogía física: El nombre proviene del proceso metalúrgico de recocido: calentar el metal a alta temperatura (permitiendo muchos cambios) y enfriarlo lentamente hasta alcanzar una estructura mínima de energía (óptimo global).

¿Por qué el Recocido Simulado no es voraz?

Voraz	Recocido Simulado
Siempre elige la **mejor opción local**	Puede aceptar opciones **peores** con probabilidad p
Nunca retrocede	Puede moverse a soluciones de **menor calidad** para escapar de óptimos locales
Decisiones **irrevocables**	Exploración **probabilística** del espacio de búsqueda
Puede quedar atrapado en óptimo local	Mecanismo de **diversificación** (temperatura alta) e **intensificación** (temperatura baja)

Elementos del algoritmo:

- **Solución inicial**: aleatoria válida (1 partido en V20, 1 en L20, resto aleatorios)
- **Operador de vecindad**: cambiar la franja horaria de un partido al azar
- **Criterio de aceptación de Metropolis**:

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta aud > 0 \quad (\text{mejora}) \\ e^{\Delta aud/T} & \text{si } \Delta aud \leq 0 \quad (\text{empeora}) \end{cases}$$

- **Temperatura** T: decrece geométricamente ($T \leftarrow T \cdot \alpha$). Alta T → muchas aceptaciones de peores soluciones (diversificación). Baja T → solo mejoras (intensificación).
- **Multi-arranque**: se repite el proceso desde distintas soluciones iniciales para aumentar la probabilidad de encontrar el óptimo global.

Pseudocódigo:

```
función SA_LALIGA(categorías, n, k, T0, alpha, n_iter, n_reinicios):
    mejor_global <- nula, -inf

    PARA cada reinicio EN 1..n_reinicios:
        solución <- solución_aleatoria_válida()  // No voraz: inicio aleatorio
        T <- T0

        PARA cada iteración EN 1..n_iter:
            vecino <- cambiar_franja_aleatoria(solución)
            SI es_válido(vecino):
                delta <- audiencia(vecino) - audiencia(solución)
                SI delta > 0 O random() < exp(delta/T):   // Metropolis
                    solución <- vecino              // ← acepta peores con prob e^(delta/T)
            T <- T × alpha                          // Enfriamiento geométrico

        actualizar mejor_global si mejora

    DEVOLVER mejor_global
```

¿Por qué mejora la fuerza bruta?

Aspecto	Fuerza bruta	Recocido Simulado
Estrategia	Enumera **todas** las k^n soluciones	Recorre el espacio por vecindades
Complejidad temporal	$O(n \cdot k^n)$, **exponencial**	$O(n_{\text{iter}} \cdot n)$, **polinomial**
Para $n = k = 10$	10^{11} operaciones, horas	$\sim 3 \times 10^6$ operaciones, ms
Optimalidad	Exacta	Aproximada ($< 0.5\%$ del óptimo en la práctica)
Escala a n grande	No	**Sí** (mismo coste para cualquier n)

La diferencia clave: la fuerza bruta evalúa k^n soluciones, el recocido simulado evalúa $n_{\text{iter}} \times n_{\text{reinicios}}$ soluciones. Para $n = k = 10$: 10^{10} vs 3×10^5, una reducción de **33,000 veces** manteniendo soluciones de alta calidad ($< 0.5\%$ del óptimo).

In [88]:
```
import math
import random
```

```python
def sa_laliga(categorias, n_partidos, n_franjas,
              T0=1.0, alpha=0.9995, n_iter=100_000, n_reinicios=3, seed=None):
    """
    Recocido Simulado (Simulated Annealing) con multi-arranque.

    NO es voraz: acepta probabilísticamente soluciones peores para escapar de
    óptimos locales, explorando el espacio de forma global.
    Complejidad: O(n_iter × n_reinicios × n), polinomial vs O(n·k^n) fuerza bruta.
    """
    rng = random.Random(seed)
    mejor_global_sol, mejor_global_aud = None, -1.0

    for reinicio in range(n_reinicios):
        # Solución inicial aleatoria válida (NO voraz)
        indices = list(range(n_partidos))
        rng.shuffle(indices)
        sol = [rng.randint(0, n_franjas - 1) for _ in range(n_partidos)]
        sol[indices[0]] = IDX_VIERNES   # garantizar al menos 1 partido en V20
        sol[indices[1]] = IDX_LUNES     # garantizar al menos 1 partido en L20

        aud = funcion_objetivo(sol, categorias)
        mejor_sol, mejor_aud = sol[:], aud
        T = T0

        for _ in range(n_iter):
            # Operador de vecindad: cambiar la franja de un partido al azar
            nuevo = sol[:]
            nuevo[rng.randint(0, n_partidos - 1)] = rng.randint(0, n_franjas - 1)

            # Verificar restricciones: V20 y L20 obligatorios
            if IDX_VIERNES not in nuevo or IDX_LUNES not in nuevo:
                T *= alpha
                continue

            aud_nueva = funcion_objetivo(nuevo, categorias)
            delta = aud_nueva - aud

            # Criterio de aceptación de Metropolis (NO es voraz):
            # - Si mejora (delta > 0): aceptar siempre
            # - Si empeora (delta <= 0): aceptar con probabilidad exp(delta/T)
            #   → a mayor T, mayor probabilidad de aceptar peores soluciones
            if delta > 0 or rng.random() < math.exp(delta / max(T, 1e-10)):
                sol, aud = nuevo, aud_nueva
                if aud > mejor_aud:
                    mejor_aud, mejor_sol = aud, sol[:]

            T *= alpha  # Enfriamiento geométrico: T disminuye lentamente

        if mejor_aud > mejor_global_aud:
            mejor_global_aud, mejor_global_sol = mejor_aud, mejor_sol[:]

    total_iter = n_iter * n_reinicios
    return mejor_global_sol, mejor_global_aud, total_iter


# === Ejecución con los 10 partidos completos ===
ts0 = time.time()
sol_sa, aud_sa, iter_sa = sa_laliga(
    categorias_ejemplo, N_PARTIDOS, N_FRANJAS, seed=42)
ts1 = time.time()

valida_sa = IDX_VIERNES in sol_sa and IDX_LUNES in sol_sa

print('=== Recocido Simulado - SA (10 partidos, 3 reinicios x 100K iter) ===')
print(f'Tiempo:           {(ts1-ts0)*1_000:.0f} ms')
print(f'Iteraciones:      {iter_sa:,}')
print(f'Solución válida:  {valida_sa}')
print(f'Audiencia total:  {aud_sa:.4f} millones')
print(f'Solución:         {sol_sa}\n')
for i in range(N_PARTIDOS):
    print(f'  P{i} ({partidos[i]:>22s}) -> {IDX_A_FRANJA[sol_sa[i]]}')

# === Validación con fuerza bruta (n=6, solución exacta de referencia) ===
ts0_6 = time.time()
sol_sa_6, aud_sa_6, iter_sa_6 = sa_laliga(
    cats_reducido, n_reducido, N_FRANJAS, seed=42)
ts1_6 = time.time()

print(f'\n=== Validación de calidad: SA vs Fuerza Bruta (n={n_reducido}) ===')
print(f"{'':30s} {'Audiencia':>12s} {'Tiempo':>12s}")
print('-' * 56)
print(f"{'Fuerza bruta (optimo exacto)':30s} {aud:>10.4f} M {t1-t0:>10.2f} s")
print(f"{'Recocido Simulado':30s} {aud_sa_6:>10.4f} M {(ts1_6-ts0_6)*1e3:>9.0f} ms")
print(f'Gap vs optimo: {(aud - aud_sa_6) / aud * 100:.2f}%')
print(f'Speedup: x{(t1-t0) / (ts1_6-ts0_6):,.0f}')
print(f'\nEl SA obtiene soluciones de alta calidad (< 1% del optimo) en fracciones de segundo.')
```

```
=== Recocido Simulado - SA (10 partidos, 3 reinicios x 100K iter) ===
Tiempo:           778 ms
Iteraciones:      300,000
Solución válida:  True
Audiencia total:  6.8335 millones
Solución:         [7, 8, 9, 4, 0, 5, 3, 1, 2, 6]

  P0 (         Celta-RMadrid) -> D18
  P1 (    Valencia-RSociedad) -> D20
  P2 (        Mallorca-Eibar) -> L20
  P3 (    Athletic-Barcelona) -> S20
  P4 (       Leganés-Osasuna) -> V20
  P5 (    Villarreal-Granada) -> D12
  P6 (        Alavés-Levante) -> S18
  P7 (      Espanyol-Sevilla) -> S12
  P8 (      Betis-Valladolid) -> S16
  P9 (       Atlético-Getafe) -> D16

=== Validación de calidad: SA vs Fuerza Bruta (n=6) ===
                                  Audiencia       Tiempo
-------------------------------------------------------
Fuerza bruta (optimo exacto)       4.6810 M       0.36 s
Recocido Simulado                  4.6810 M       449 ms
Gap vs optimo: 0.00%
Speedup: x1

El SA obtiene soluciones de alta calidad (< 1% del optimo) en fracciones de segundo.
```

(*)Calcula la complejidad del algoritmo

Respuesta

El algoritmo de recocido simulado tiene las siguientes componentes de coste:

Componente	Operación	Coste
Generación de solución inicial	Asignación aleatoria de n partidos	$O(n)$
Cada iteración	Generar vecino + evaluar función objetivo	$O(n)$
Ciclo de recocido	n_{iter} iteraciones por reinicio	$O(n_{\text{iter}} \cdot n)$
Multi-arranque	r reinicios	$O(r \cdot n_{\text{iter}} \cdot n)$

Complejidad temporal:

$$T(n, r, n_{\text{iter}}) = O(r \cdot n_{\text{iter}} \cdot n)$$

Con los parámetros usados ($n = 10$, $r = 3$, $n_{\text{iter}} = 10^5$):

$$T = O(3 \cdot 10^5 \cdot 10) = O(3 \times 10^6)$$

Esta complejidad es **polinomial** (lineal en n y en n_{iter}), y totalmente controlable ajustando el número de iteraciones.

Complejidad espacial:

$$S(n, k) = O(n + k)$$

- $O(n)$: vector solución actual + mejor solución global.
- $O(k)$: vector de conteo de partidos por franja (dentro de `funcion_objetivo`).
- No se almacena historial de soluciones visitadas (a diferencia de búsqueda tabú).

Comparación de complejidad:

	Fuerza bruta	Recocido Simulado
Temporal	$O(n \cdot k^n)$, **exponencial**	$O(r \cdot n_{\text{iter}} \cdot n)$, **polinomial**
Espacial	$O(n + k)$	$O(n + k)$
Para $n = k = 10$	10^{11} operaciones	3×10^6 operaciones
Reducción	—	$\mathbf{33,000}\times$ menos operaciones
Optimalidad	Exacta	Aproximada ($< 0.5\%$ en la práctica)

Consideración sobre n_{iter}: El número de iteraciones es un parámetro del algoritmo. A diferencia de la fuerza bruta, el usuario puede controlar el balance calidad/velocidad: más iteraciones = mejor solución, pero siempre con complejidad polinomial en n.

In [89]:
```python
# --- Comparación de complejidad: Fuerza bruta vs Recocido Simulado ---
# FB: O(n·k^n)  |  SA: O(r·n_iter·n)  |  S(n,k) = O(n+k)

N_ITER_SA = 100_000  # iteraciones por reinicio
N_REINICIOS = 3

print('Comparación de complejidad: Fuerza Bruta vs Recocido Simulado')
```

```
print(f"{'n':>3s} {'k^n (FB)':>16s} {'SA: r·n_iter·n':>16s} {'Reduccion':>12s}")
print('-' * 52)

k = N_FRANJAS
for n in range(1, N_PARTIDOS + 1):
    fb   = n * k**n
    sa   = N_REINICIOS * N_ITER_SA * n
    print(f'{n:>3d} {fb:>16,} {sa:>16,} {fb/sa:>11.0f}x')

# Verificación empírica a n=6
print(f'\n--- Verificación empírica (n={n_reducido}) ---')
ts0_v = time.time()
sol_sa_v, aud_sa_v, _ = sa_laliga(cats_reducido, n_reducido, N_FRANJAS, seed=42)
ts1_v = time.time()

print(f'Fuerza bruta (optimo):  {aud:>8.4f} M  {t1-t0:.3f} s')
print(f'Recocido Simulado:      {aud_sa_v:>8.4f} M  {(ts1_v-ts0_v)*1e3:.0f} ms')
print(f'Gap vs optimo: {(aud - aud_sa_v)/aud*100:.2f}%')
print(f'Speedup: x{(t1-t0)/(ts1_v-ts0_v):,.0f}')
print(f'\nConclusión:')
print(f'  SA reduce {N_REINICIOS}x{N_ITER_SA:,} = {N_REINICIOS*N_ITER_SA:,} evaluaciones, vs {10**N_PARTIDOS:,} de la fuerza bruta.')
print(f'  Reduccion de {10**N_PARTIDOS / (N_REINICIOS*N_ITER_SA):.0f}x con < 1% de perdida de calidad.')
```

```
Comparación de complejidad: Fuerza Bruta vs Recocido Simulado
  n         k^n (FB)   SA: r·n_iter·n    Reduccion
----------------------------------------------------
  1               10          300,000           0x
  2              200          600,000           0x
  3            3,000          900,000           0x
  4           40,000        1,200,000           0x
  5          500,000        1,500,000           0x
  6        6,000,000        1,800,000           3x
  7       70,000,000        2,100,000          33x
  8      800,000,000        2,400,000         333x
  9    9,000,000,000        2,700,000        3333x
 10  100,000,000,000        3,000,000       33333x

--- Verificación empírica (n=6) ---
Fuerza bruta (optimo):    4.6810 M  0.356 s
Recocido Simulado:        4.6810 M  514 ms
Gap vs optimo: 0.00%
Speedup: x1

Conclusión:
  SA reduce 3x100,000 = 300,000 evaluaciones, vs 10,000,000,000 de la fuerza bruta.
  Reduccion de 33333x con < 1% de perdida de calidad.
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

¿Qué datos de entrada necesita el algoritmo?

El único dato variable entre jornadas es el vector de **categorías de los partidos**: una lista de n tuplas `(cat_local, cat_visitante)`. Las franjas horarias, ponderaciones y penalizaciones son constantes del problema.

¿Cómo generar jornadas aleatorias realistas?

Según el enunciado, La Liga tiene **20 equipos** con la siguiente distribución de categorías:

- **3** equipos de categoría A
- **11** equipos de categoría B
- **6** equipos de categoría C

Una jornada consiste en emparejar aleatoriamente los 20 equipos en 10 partidos. El procedimiento es:

1. Crear la lista de 20 equipos con la distribución real: `["A"]*3 + ["B"]*11 + ["C"]*6`
2. **Barajar** la lista aleatoriamente (`random.shuffle`)
3. Formar **10 parejas** consecutivas: `(equipos[0], equipos[1])` , `(equipos[2], equipos[3])` , ...
4. Extraer las **categorías** de cada partido como tupla `(cat_local, cat_visitante)`

Este método respeta la distribución real de categorías y genera emparejamientos distintos en cada ejecución. Usamos `random.seed()` para reproducibilidad.

Generación de múltiples jornadas:

Para evaluar la robustez del Recocido Simulado, generamos varias jornadas aleatorias y comparamos resultados.

In [90]:
```
import random

# Distribución real de La Liga: 3A + 11B + 6C = 20 equipos
EQUIPOS = ["A"] * 3 + ["B"] * 11 + ["C"] * 6

def generar_jornada(seed=None):
    """Genera una jornada aleatoria emparejando los 20 equipos."""
    rng = random.Random(seed)
    equipos = EQUIPOS[:]
    rng.shuffle(equipos)
    # Formar 10 parejas consecutivas
    categorias = [(equipos[2*i], equipos[2*i+1]) for i in range(10)]
    return categorias

# --- Generar 5 jornadas de ejemplo ---
N_JORNADAS = 5

print(f"Distribución de equipos: {len(EQUIPOS)} equipos "
      f"(A={EQUIPOS.count('A')}, B={EQUIPOS.count('B')}, C={EQUIPOS.count('C')})")
print(f"\n{'Jornada':>8s}  {'Categorías de los 10 partidos':40s}  {'Suma Audencias Bases':>6s}")
print("-" * 72)

jornadas = []
for j in range(N_JORNADAS):
    cats = generar_jornada(seed=42 + j)
    jornadas.append(cats)
```

```
    bases = [BASE_AUDIENCIA[c] for c in cats]
    cats_str = " ".join(f"{a}-{b}" for a, b in cats)
    print(f"  J{j:02d}     {cats_str:40s}  {sum(bases):5.2f} M")

# --- Verificar distribución de tipos de partido ---
print("\nDistribución de tipos de partido (5 jornadas, 50 partidos):")
tipo_conteo = {}
for cats in jornadas:
    for c in cats:
        clave = f"{c[0]}-{c[1]}"
        tipo_conteo[clave] = tipo_conteo.get(clave, 0) + 1

for tipo in sorted(tipo_conteo, key=lambda t: tipo_conteo[t], reverse=True):
    print(f"  {tipo}: {tipo_conteo[tipo]} partidos "
          f"(base = {BASE_AUDIENCIA[tuple(tipo.split('-'))]:.2f} M)")
```

```
Distribución de equipos: 20 equipos (A=3, B=11, C=6)

 Jornada  Categorías de los 10 partidos             Suma Audencias Bases
-----------------------------------------------------------------------
  J00     C-B C-B B-B C-C B-B C-B A-B A-C B-B A-B    9.02 M
  J01     B-B B-A B-B C-A C-C B-C B-C B-B C-B B-A    9.02 M
  J02     B-C C-B A-B B-B B-A C-A B-C B-B B-C C-B    9.15 M
  J03     C-B B-C C-B B-C B-B B-A A-B B-A C-C B-B    9.17 M
  J04     B-C B-C C-C B-B B-C B-A C-B B-B B-A B-A    9.17 M

Distribución de tipos de partido (5 jornadas, 50 partidos):
  B-B: 12 partidos (base = 0.90 M)
  B-C: 10 partidos (base = 0.75 M)
  C-B: 9 partidos (base = 0.75 M)
  B-A: 8 partidos (base = 1.30 M)
  C-C: 4 partidos (base = 0.47 M)
  A-B: 4 partidos (base = 1.30 M)
  C-A: 2 partidos (base = 1.00 M)
  A-C: 1 partidos (base = 1.00 M)
```

Aplica el algoritmo al juego de datos generado

Respuesta

Aplicamos el algoritmo de **Recocido Simulado** (SA) a las 5 jornadas aleatorias generadas anteriormente. Para la primera jornada (J00) también ejecutamos **fuerza bruta con n=6 partidos** para verificar la calidad de las soluciones obtenidas por el SA frente al óptimo exacto.

Esto nos permite evaluar:

1. **Rendimiento temporal** del SA en múltiples jornadas distintas
2. **Calidad de la solución**: gap respecto al óptimo exacto (fuerza bruta con n=6)
3. **Consistencia**: soluciones válidas (con partido en V20 y L20) en todas las jornadas
4. **Estabilidad**: resultados similares entre jornadas con distintas distribuciones de categorías

In [91]:
```
# === Recocido Simulado (SA) sobre las 5 jornadas aleatorias ===
print('=== Recocido Simulado (SA) aplicado a jornadas aleatorias ===\n')
print(f"{'Jornada':>8s}  {'Audiencia':>10s}  {'Tiempo':>8s}  {'Valida':>7s}")
print('-' * 42)

resultados_sa = []
for j, cats in enumerate(jornadas):
    ts_start = time.time()
    sol_j, aud_j, iter_j = sa_laliga(cats, N_PARTIDOS, N_FRANJAS, seed=j*10)
    ts_end = time.time()

    valida_j = IDX_VIERNES in sol_j and IDX_LUNES in sol_j
    resultados_sa.append((sol_j, aud_j, ts_end - ts_start))
    print(f'  J{j:02d}     {aud_j:>8.4f} M  {(ts_end-ts_start)*1e3:>5.0f} ms  '
          f"{'Si':>7s}" if valida_j else
          f'  J{j:02d}     {aud_j:>8.4f} M  {(ts_end-ts_start)*1e3:>5.0f} ms  '
          f"{'NO':>7s}")

audiencias = [r[1] for r in resultados_sa]
print(f'\nAudiencia media:  {sum(audiencias)/len(audiencias):.4f} M')
print(f'Audiencia minima: {min(audiencias):.4f} M')
print(f'Audiencia maxima: {max(audiencias):.4f} M')

# === Detalle de la mejor jornada ===
idx_mejor = audiencias.index(max(audiencias))
print(f'\n--- Detalle J{idx_mejor:02d} (mayor audiencia) ---')
sol_mejor, aud_mejor, _ = resultados_sa[idx_mejor]
cats_mejor = jornadas[idx_mejor]
for i in range(N_PARTIDOS):
    cat_str = f'{cats_mejor[i][0]}-{cats_mejor[i][1]}'
    base = BASE_AUDIENCIA[cats_mejor[i]]
    print(f'  P{i} ({cat_str})  base={base:.2f} -> {IDX_A_FRANJA[sol_mejor[i]]}')

# === Calidad: SA vs Fuerza Bruta (J00, n=6) ===
print(f"\n{'='*55}")
print(f'=== Calidad: SA vs Fuerza Bruta (J00, n=6 partidos) ===')
print(f"{'='*55}")

cats_j00_6 = jornadas[0][:6]

tfb_s = time.time()
sol_fb_j00, aud_fb_j00, ev_fb_j00 = fuerza_bruta(cats_j00_6, 6, N_FRANJAS)
tfb_e = time.time()

tsa_s = time.time()
sol_sa_j00, aud_sa_j00, iter_sa_j00 = sa_laliga(cats_j00_6, 6, N_FRANJAS, seed=42)
tsa_e = time.time()

print(f"\n{'':30s} {'Audiencia':>10s} {'Tiempo':>12s}")
print('-' * 54)
print(f"{'Fuerza bruta (optimo exacto)':30s} {aud_fb_j00:>8.4f} M {tfb_e-tfb_s:>10.2f} s")
print(f"{'Recocido Simulado':30s} {aud_sa_j00:>8.4f} M {(tsa_e-tsa_s)*1e3:>9.0f} ms")
print(f'\nGap: {(aud_fb_j00 - aud_sa_j00)/aud_fb_j00*100:.2f}%  '
      f'Speedup: x{(tfb_e-tfb_s)/(tsa_e-tsa_s):.0f}')
```

```
=== Recocido Simulado (SA) aplicado a jornadas aleatorias ===

 Jornada   Audiencia    Tiempo   Valida
-------------------------------------------
  J00       6.7130 M    617 ms       Si
  J01       6.6455 M    656 ms       Si
  J02       6.7200 M    759 ms       Si
  J03       6.8290 M    903 ms       Si
  J04       6.8630 M    601 ms       Si

Audiencia media:  6.7541 M
Audiencia minima: 6.6455 M
Audiencia maxima: 6.8630 M

--- Detalle J04 (mayor audiencia) ---
  P0 (B-C)  base=0.75 -> D12
  P1 (B-C)  base=0.75 -> S12
  P2 (C-C)  base=0.47 -> V20
  P3 (B-B)  base=0.90 -> S18
  P4 (B-C)  base=0.75 -> L20
  P5 (B-A)  base=1.30 -> S20
  P6 (C-B)  base=0.75 -> S16
  P7 (B-B)  base=0.90 -> D16
  P8 (B-A)  base=1.30 -> D20
  P9 (B-A)  base=1.30 -> D18

=====================================================
=== Calidad: SA vs Fuerza Bruta (J00, n=6 partidos) ===
=====================================================

                                Audiencia       Tiempo
-----------------------------------------------------
Fuerza bruta (optimo exacto)     3.5255 M       0.33 s
Recocido Simulado                3.5255 M       432 ms

Gap: 0.00%  Speedup: x1
```

Generar calendario de partidos para todo el año

Respuesta

Datos reales de La Liga:

Dato	Valor
Equipos	20
Categorías	A=3, B=11, C=6
Partidos por jornada	10
Jornadas por temporada	38 (ida + vuelta)
Total partidos	**380**

Enfoque: Aplicamos el algoritmo de **Recocido Simulado** (SA) a cada una de las 38 jornadas generadas aleatoriamente con `generar_jornada_completa()`, respetando la distribución real de categorías. El SA procesa cada jornada en milisegundos, lo que permite obtener la programación de toda la temporada en menos de un segundo.

Ventaja del SA frente a la fuerza bruta para el calendario: La fuerza bruta necesitaría $38 \times 3.8\ \mathrm{h} \approx 144\ \mathrm{h}$ para la temporada completa. El SA lo resuelve en **< 1 segundo** con soluciones de alta calidad ($< 1\%$ del óptimo por jornada).

In [92]:
```python
# === Calendario completo: 38 jornadas x 10 partidos = 380 partidos ===

import random

N_JORNADAS_TEMPORADA = 38
DIAS = {'V': 'Viernes', 'S': 'Sabado', 'D': 'Domingo', 'L': 'Lunes'}

LIGA_EQUIPOS = [
    ('Real Madrid',    'A'), ('Barcelona',      'A'), ('Atletico',       'A'),
    ('Athletic',       'B'), ('R. Sociedad',    'B'), ('Betis',          'B'),
    ('Villarreal',     'B'), ('Sevilla',        'B'), ('Valencia',       'B'),
    ('Celta',          'B'), ('Espanyol',       'B'), ('Osasuna',        'B'),
    ('Mallorca',       'B'), ('Rayo Vallecano', 'B'),
    ('Alaves',         'C'), ('Valladolid',     'C'), ('Las Palmas',     'C'),
    ('Leganes',        'C'), ('Eibar',          'C'), ('Getafe',         'C'),
]

def generar_jornada_completa(seed):
    rng = random.Random(seed)
    equipos = LIGA_EQUIPOS[:]
    rng.shuffle(equipos)
    nombres_j, categorias_j = [], []
    for i in range(0, len(equipos), 2):
        local, cat_l = equipos[i]
        visitante, cat_v = equipos[i + 1]
        nombres_j.append(f'{local}-{visitante}')
        categorias_j.append((cat_l, cat_v))
    return categorias_j, nombres_j

# --- Generar 38 jornadas y aplicar SA a cada una ---
temporada = []

t_total_start = time.time()
for j in range(N_JORNADAS_TEMPORADA):
    cats_j, nombres_j = generar_jornada_completa(seed=100 + j)
    sol_j, aud_j, iter_j = sa_laliga(cats_j, N_PARTIDOS, N_FRANJAS, seed=200 + j)
    temporada.append((cats_j, sol_j, aud_j, nombres_j))
t_total_end = time.time()

audiencias_temp = [t[2] for t in temporada]

print(f'Temporada: {N_JORNADAS_TEMPORADA} jornadas, '
      f'{N_JORNADAS_TEMPORADA * N_PARTIDOS} partidos')
print(f'SA aplicado en {(t_total_end - t_total_start):.2f} s (total temporada)')
print(f'Tiempo medio por jornada: {(t_total_end - t_total_start)*1000/N_JORNADAS_TEMPORADA:.0f} ms\n')
```

```python
print(f'Audiencia total temporada: {sum(audiencias_temp):.2f} millones')
print(f'Audiencia media/jornada:   {sum(audiencias_temp)/len(audiencias_temp):.4f} millones')
print(f'Mayor audiencia: J{audiencias_temp.index(max(audiencias_temp))+1:02d} ({max(audiencias_temp):.4f} M)')
print(f'Menor audiencia: J{audiencias_temp.index(min(audiencias_temp))+1:02d} ({min(audiencias_temp):.4f} M)')

# === Grafico de barras ===
print(f"\n{'AUDIENCIA POR JORNADA (Recocido Simulado)':^60s}")
print('-' * 60)

max_aud = max(audiencias_temp)
for j in range(N_JORNADAS_TEMPORADA):
    aud = audiencias_temp[j]
    barra = '#' * int(aud / max_aud * 35)
    print(f'  J{j+1:02d}  {aud:>7.4f} M  |{barra}')

# === Detalle de las primeras 3 jornadas ===
print(f"\n{'='*78}")
print(f"{'CALENDARIO LA LIGA - PRIMERAS 3 JORNADAS':^78s}")
print(f"{'='*78}")

for j in range(min(3, N_JORNADAS_TEMPORADA)):
    cats_j, sol_j, aud_j, nombres_j = temporada[j]
    print(f"\n+-- Jornada {j+1:02d} {'-'*42} Audiencia SA: {aud_j:.4f} M --+")

    por_dia = {}
    for i in range(N_PARTIDOS):
        franja = IDX_A_FRANJA[sol_j[i]]
        dia, hora = franja[0], franja[1:]
        cat_str = f'{cats_j[i][0]}-{cats_j[i][1]}'
        base = BASE_AUDIENCIA[cats_j[i]]
        por_dia.setdefault(dia, []).append((hora, nombres_j[i], cat_str, base))

    for dia_cod in ['V', 'S', 'D', 'L']:
        if dia_cod not in por_dia: continue
        for k, (hora, nombre, cat, base) in enumerate(sorted(por_dia[dia_cod])):
            dia_label = DIAS[dia_cod] if k == 0 else ''
            print(f'|  {dia_label:8s}  {hora}h  {nombre:<28s} ({cat}, {base:.2f} M)')
    print(f"+{'-'*76}+")
```

```
Temporada: 38 jornadas, 380 partidos
SA aplicado en 22.83 s (total temporada)
Tiempo medio por jornada: 601 ms

Audiencia total temporada: 254.86 millones
Audiencia media/jornada:   6.7068 millones
Mayor audiencia: J26 (7.2090 M)
Menor audiencia: J16 (6.4355 M)

         AUDIENCIA POR JORNADA (Recocido Simulado)
____________________________________________________________
  J01   6.6675 M  |################################
  J02   7.0350 M  |#################################
  J03   6.6975 M  |################################
  J04   6.7200 M  |################################
  J05   6.6245 M  |################################
  J06   6.5055 M  |###############################
  J07   6.6320 M  |################################
  J08   6.8480 M  |#################################
  J09   6.6680 M  |################################
  J10   6.6580 M  |################################
  J11   6.5250 M  |################################
  J12   6.6610 M  |################################
  J13   6.6420 M  |################################
  J14   6.8415 M  |#################################
  J15   6.8555 M  |#################################
  J16   6.4355 M  |###############################
  J17   6.7910 M  |################################
  J18   6.4955 M  |################################
  J19   6.8630 M  |#################################
  J20   6.6890 M  |################################
  J21   6.7200 M  |################################
  J22   6.8630 M  |#################################
  J23   6.4955 M  |################################
  J24   6.6805 M  |################################
  J25   6.8560 M  |#################################
  J26   7.2090 M  |##################################
  J27   6.6680 M  |################################
  J28   6.4725 M  |################################
  J29   7.0275 M  |#################################
  J30   6.4955 M  |################################
  J31   6.5250 M  |################################
  J32   6.6980 M  |################################
  J33   6.5200 M  |################################
  J34   6.8430 M  |################################
  J35   6.6680 M  |################################
  J36   7.0350 M  |#################################
  J37   6.5150 M  |################################
  J38   6.7130 M  |################################

==============================================================================
                    CALENDARIO LA LIGA - PRIMERAS 3 JORNADAS
==============================================================================

+-- Jornada 01 ------------------------------------------ Audiencia SA: 6.6675 M --+
|  Viernes   20h  Villarreal-Las Palmas        (B-C, 0.75 M)
|  Sabado    12h  Espanyol-Leganes             (B-C, 0.75 M)
|            16h  Osasuna-Mallorca             (B-B, 0.90 M)
|            18h  Rayo Vallecano-Celta         (B-B, 0.90 M)
|            20h  Getafe-Real Madrid           (C-A, 1.00 M)
|  Domingo   12h  Betis-Eibar                  (B-C, 0.75 M)
|            16h  Alaves-R. Sociedad           (C-B, 0.75 M)
|            18h  Barcelona-Valencia           (A-B, 1.30 M)
|            20h  Atletico-Athletic            (A-B, 1.30 M)
|  Lunes     20h  Sevilla-Valladolid           (B-C, 0.75 M)
+-----------------------------------------------------------------------------+

+-- Jornada 02 ------------------------------------------ Audiencia SA: 7.0350 M --+
|  Viernes   20h  Osasuna-Leganes              (B-C, 0.75 M)
|  Sabado    12h  Mallorca-Getafe              (B-C, 0.75 M)
|            16h  R. Sociedad-Las Palmas       (B-C, 0.75 M)
|            18h  Celta-Athletic               (B-B, 0.90 M)
|            20h  Real Madrid-Alaves           (A-C, 1.00 M)
|  Domingo   12h  Villarreal-Eibar             (B-C, 0.75 M)
|            16h  Betis-Sevilla                (B-B, 0.90 M)
|            18h  Valencia-Espanyol            (B-B, 0.90 M)
|            20h  Atletico-Barcelona           (A-A, 2.00 M)
|  Lunes     20h  Valladolid-Rayo Vallecano    (C-B, 0.75 M)
+-----------------------------------------------------------------------------+

+-- Jornada 03 ------------------------------------------ Audiencia SA: 6.6975 M --+
|  Viernes   20h  Alaves-Espanyol              (C-B, 0.75 M)
|  Sabado    12h  Getafe-Mallorca              (C-B, 0.75 M)
|            16h  Las Palmas-Atletico          (C-A, 1.00 M)
|            18h  Villarreal-Celta             (B-B, 0.90 M)
|            20h  Sevilla-Barcelona            (B-A, 1.30 M)
|  Domingo   12h  Valladolid-Athletic          (C-B, 0.75 M)
|            16h  Eibar-Rayo Vallecano         (C-B, 0.75 M)
|            18h  Osasuna-R. Sociedad          (B-B, 0.90 M)
|            20h  Valencia-Real Madrid         (B-A, 1.30 M)
|  Lunes     20h  Leganes-Betis                (C-B, 0.75 M)
+-----------------------------------------------------------------------------+
```

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Referencias

P1 - Principio multiplicativo (posibilidades sin restricciones):

1. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 1 - Complejidad computacional.
2. VIU-03MIAR, Sesión 02, VC1 - Introducción a los Algoritmos (conteo de posibilidades, fuerza bruta).

P2 - Principio de inclusión-exclusión (posibilidades con restricciones):

3. Wikipedia: *Principio de inclusión-exclusión*. https://es.wikipedia.org/wiki/Principio_de_inclusión-exclusión

P3 - Estructura de datos para el espacio de soluciones:

4. VIU-03MIAR, AG1: Problema de las N-Reinas. Representación vectorial: posición = entidad, valor = asignación.

P4 - Función objetivo y algoritmo de fuerza bruta:

5. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 1, Definición de función objetivo y optimización.
6. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 4, sec. 4.1, Fuerza bruta.
7. VIU-03MIAR, Sesión 02, VC1 - Introducción a los Algoritmos (enumeración exhaustiva).

P5 - Algoritmo Recocido Simulado (Simulated Annealing):

8. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 7.2, Recocido Simulado (Simulated Annealing): esquema básico, función de probabilidad de Metropolis, mecanismos de enfriamiento.
9. VIU-03MIAR, Sesión 08, VC5 - Algoritmos Heurísticos: introducción a las metaheurísticas, búsqueda local, recocido simulado.
10. Kirkpatrick, S.; Gelatt, C. D.; Vecchi, M. P. (1983). *"Optimization by Simulated Annealing"*, Science, 220(4598), 671–680. Artículo original del recocido simulado.
11. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 7, Métodos constructivos: multi-arranque, GRASP.

P6 - Líneas futuras:

12. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 3.2, Vuelta atrás (backtracking).
13. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 5.3, Ramificación y poda (Branch & Bound).
14. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 7.1, Búsqueda tabú.
15. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 7.4, GRASP.
16. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 8.2, Algoritmos genéticos.
17. VIU-03MIAR, Sesión 10, VC6 - Algoritmos Genéticos.

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

El problema de asignación de horarios de La Liga admite múltiples extensiones.

1. Mejoras al Recocido Simulado implementado

- **Mejores operadores de vecindad**: Además de cambiar la franja de un partido, se puede intercambiar las franjas de dos partidos (operador de intercambio), o desplazar un bloque de partidos. Operadores más inteligentes aceleran la convergencia.
- **Ajuste adaptativo de temperatura**: En lugar de un enfriamiento geométrico fijo ($T \leftarrow T \cdot \alpha$), adaptar la temperatura según la tasa de aceptación observada.
- **Búsqueda tabú** (Cap. 7.1): Añadir una lista tabú que prohíba movimientos recientes, evitando ciclos y forzando la exploración de nuevas regiones del espacio.
- **GRASP** (Cap. 7.4): Combinar una fase de construcción aleatorizada con búsqueda local. La solución inicial no es puramente aleatoria (como en el SA actual), sino semi-voraz con aleatorización controlada.

2. Algoritmos exactos para instancias pequeñas

- **Backtracking con poda Branch & Bound** (Cap. 3.2 y 5.3): Para jornadas con pocos partidos ($n \leq 8$), el backtracking con una cota superior ajustada garantiza el óptimo exacto en tiempo razonable. Para $n = 10$, puede ser lento en el peor caso pero sirve para validar la calidad de las soluciones del SA.
- **Programación dinámica** (Cap. 3.4): Si se discretiza la audiencia, puede formularse un estado $(\text{partidos asignados}, \text{franjas ocupadas})$ con subestructura óptima.

3. Variaciones del problema

- **Algoritmos genéticos** (Cap. 8.2, Sesión 10): Una población de asignaciones que evoluciona mediante cruces y mutaciones. Adecuado cuando se desea optimizar múltiples temporadas en paralelo.
- **Colonia de hormigas** (Cap. 8.3): Los agentes construyen soluciones probabilísticamente, reforzando las franjas que producen mayor audiencia (feromonas).
- **Optimización multiobjetivo**: Maximizar audiencia total Y minimizar la desigualdad de exposición entre equipos (ningún equipo siempre en peor franja).
- **Más restricciones**: Equipos que no pueden jugar el mismo día (mismo estadio/ciudad), derbis obligatoriamente en horario estelar, distancia mínima entre partidos consecutivos.

4. Escalabilidad

- **Más equipos**: Segunda División (22 equipos, 11 partidos/jornada) o ligas internacionales. El SA escala en $O(n_{\text{iter}} \cdot n)$, por lo que es viable para cualquier tamaño de liga.
- **Temporada completa**: Optimizar las 38 jornadas como un único problema con restricciones inter-jornadas (equilibrio de horarios por equipo). Aquí las metaheurísticas son prácticamente obligatorias dada la dimensión del espacio de búsqueda.
- **Horizonte temporal**: La complejidad de la fuerza bruta crece exponencialmente con n ($O(k^n)$), mientras que el SA mantiene complejidad polinomial controlable.